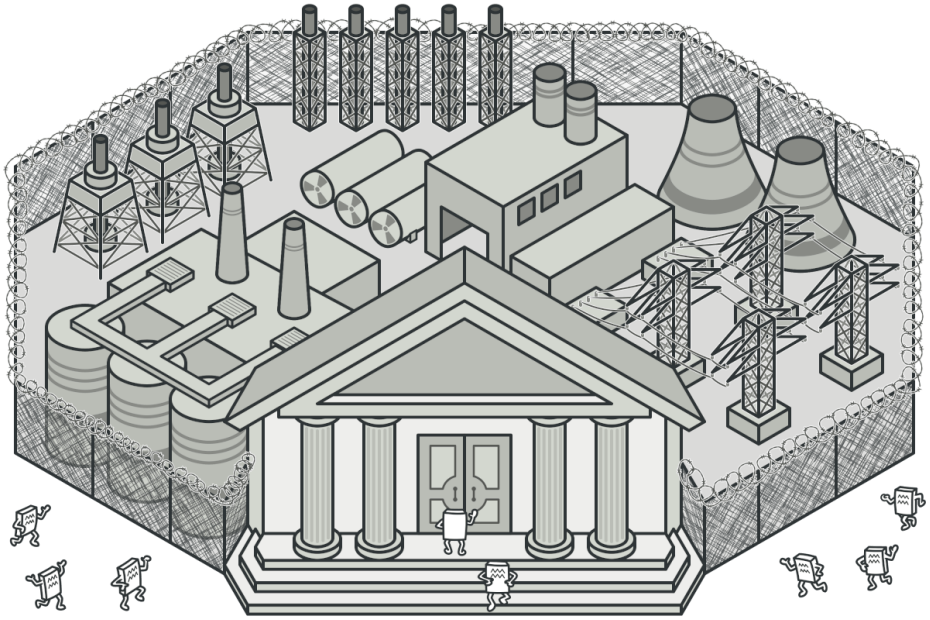
Структурные паттерны. Фасад



Паттерн Фасад
Facade



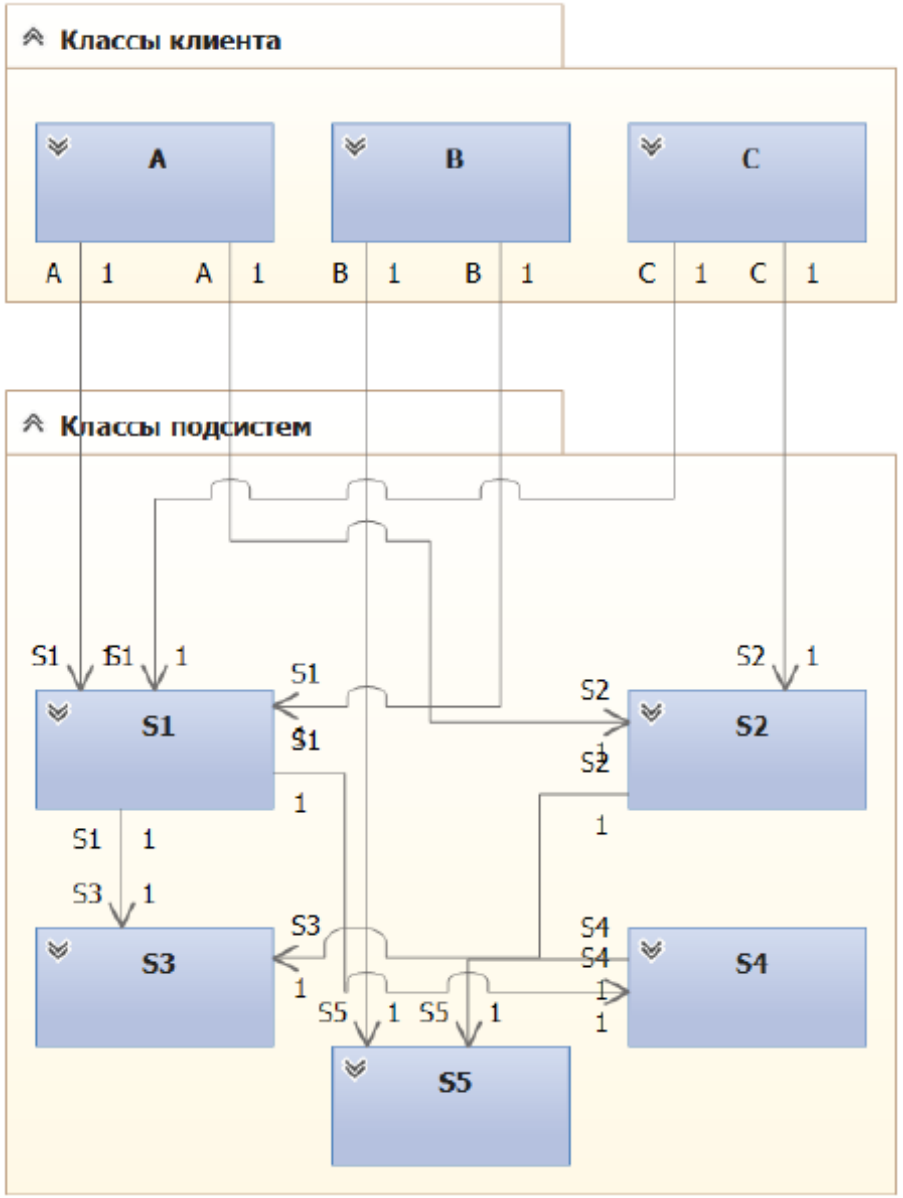
Фасад — это структурный паттерн, который предоставляет простой (но урезанный) интерфейс к сложной системе объектов, библиотеке или фреймворку.

Паттерн Facade - предоставляет унифицированный интерфейс (набор имен методов) вместо интерфейса некоторой подсистемы (набора взаимосвязанных классов или объектов).

Класс Facade предоставляет высокоуровневый интерфейс (набор методов одного класса) вместо низкоуровневого интерфейса подсистемы (наборов методов из разных классов входящих в состав подсистемы).

Проблема

- Клиентскому коду приходится работать с большим количеством объектов некой сложной библиотеки или фреймворка.
- Необходимо инициализировать эти объекты, следить за правильным порядком зависимостей и т.д.
- В результате логика приложения тесно переплетается с деталями реализации сторонних классов.
- Такой код довольно сложно понимать и поддерживать.
- Главная задача, решаемая при проектировании – это сокращение количества связей отношений между классами т.е.– ***уменьшение зависимостей классов друг от друга***.

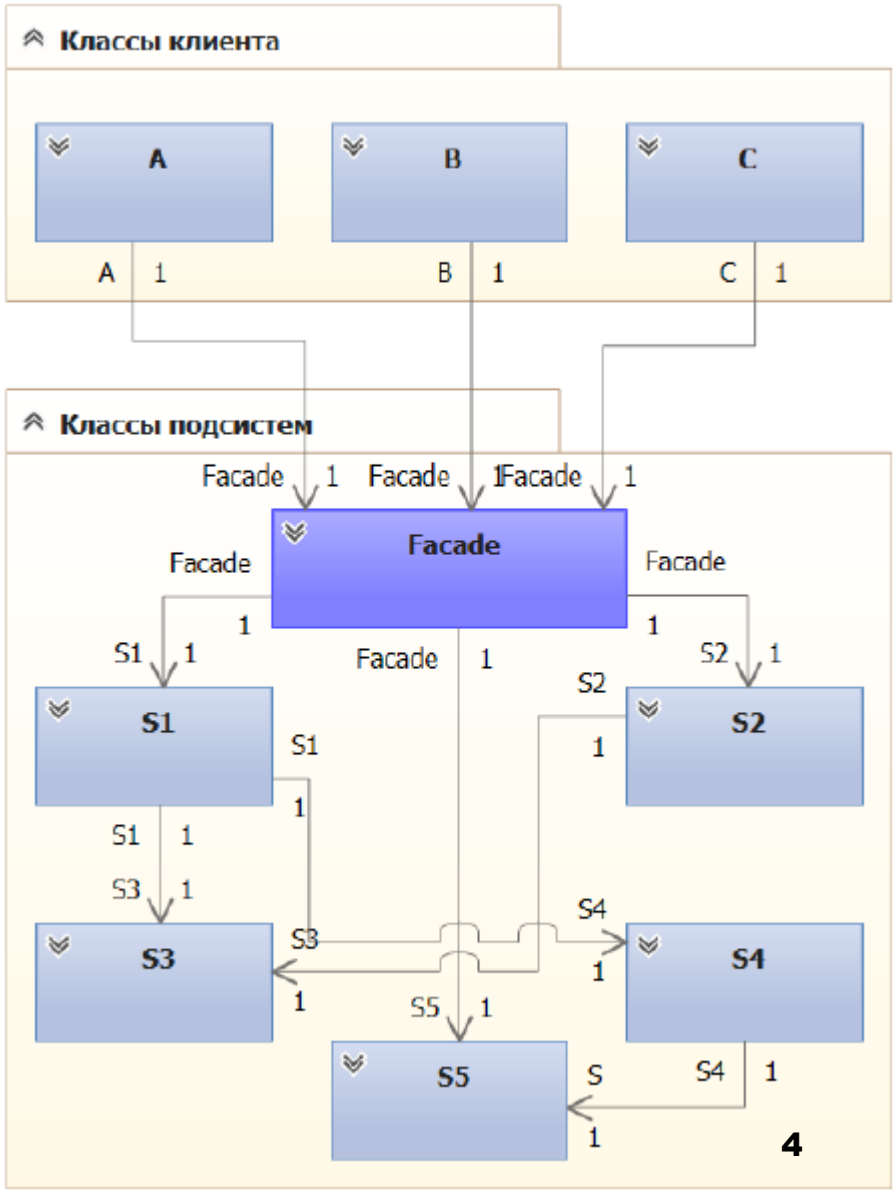


Решение

Вводится дополнительный ассоциативный класс **Facade**, через единый и упрощенный интерфейс которого происходит взаимодействие с классами подсистем

Фасад — это простой интерфейс для работы со сложной подсистемой, содержащей множество классов.

Фасад может иметь урезанный интерфейс, не имеющий 100% функциональности, которой можно достичь, используя сложную подсистему напрямую. Но он предоставляет именно те свойства, которые нужны клиенту, и скрывает все остальные.

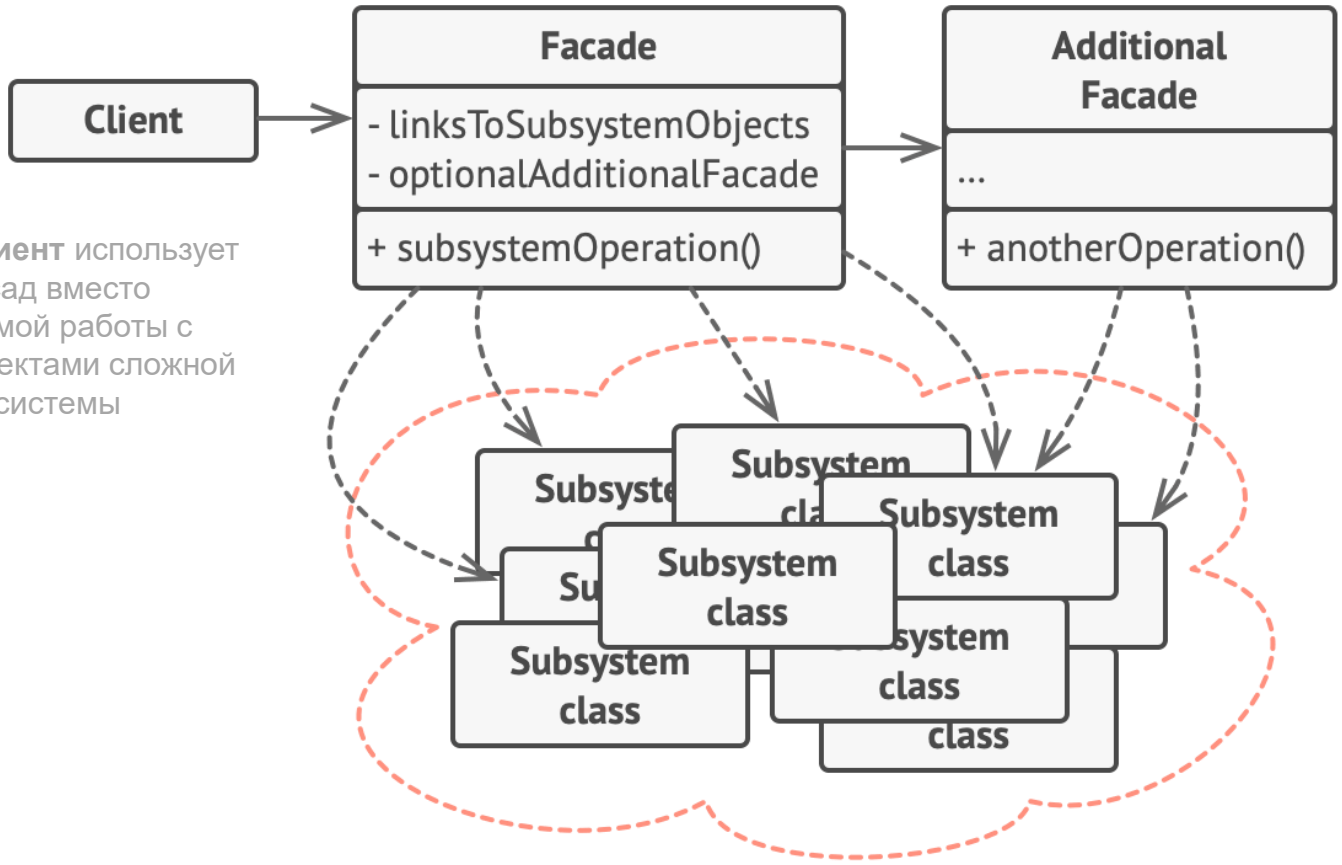


мои документы
государственные и муниципальные услуги

Структура паттерна

[1]**Фасад** предоставляет быстрый доступ к определённой функциональности подсистемы. Он «знает», каким классам нужно переадресовать запрос, и какие данные для этого нужны.

[2]**Дополнительный фасад** можно ввести, чтобы не «захламлять» единственный фасад разнородной функциональностью. Он может использоваться как клиентом, так и другими фасадами.



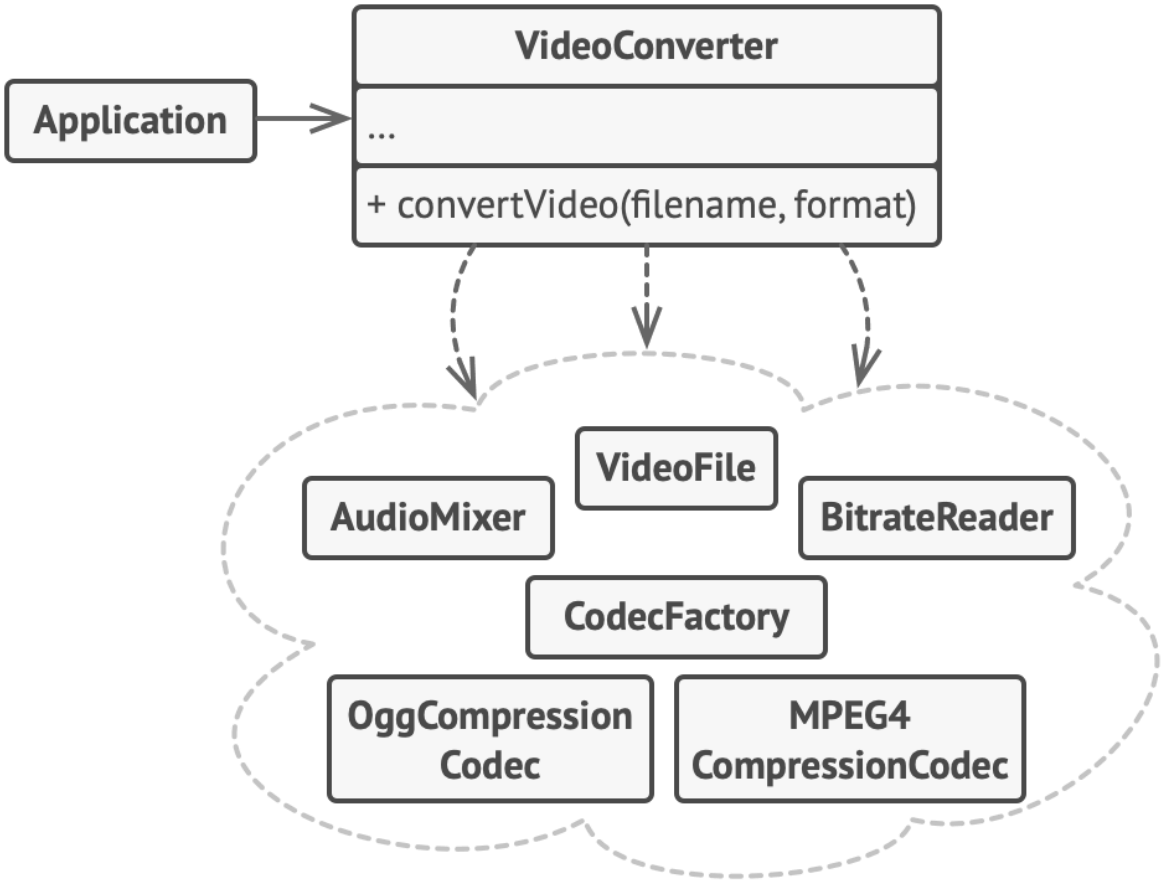
[4]**Клиент** использует фасад вместо прямой работы с объектами сложной подсистемы

[3]**Сложная подсистема** состоит из множества разнообразных классов. Для того, чтобы заставить их что-то делать, нужно знать подробности устройства подсистемы, порядок инициализации объектов и т.д.

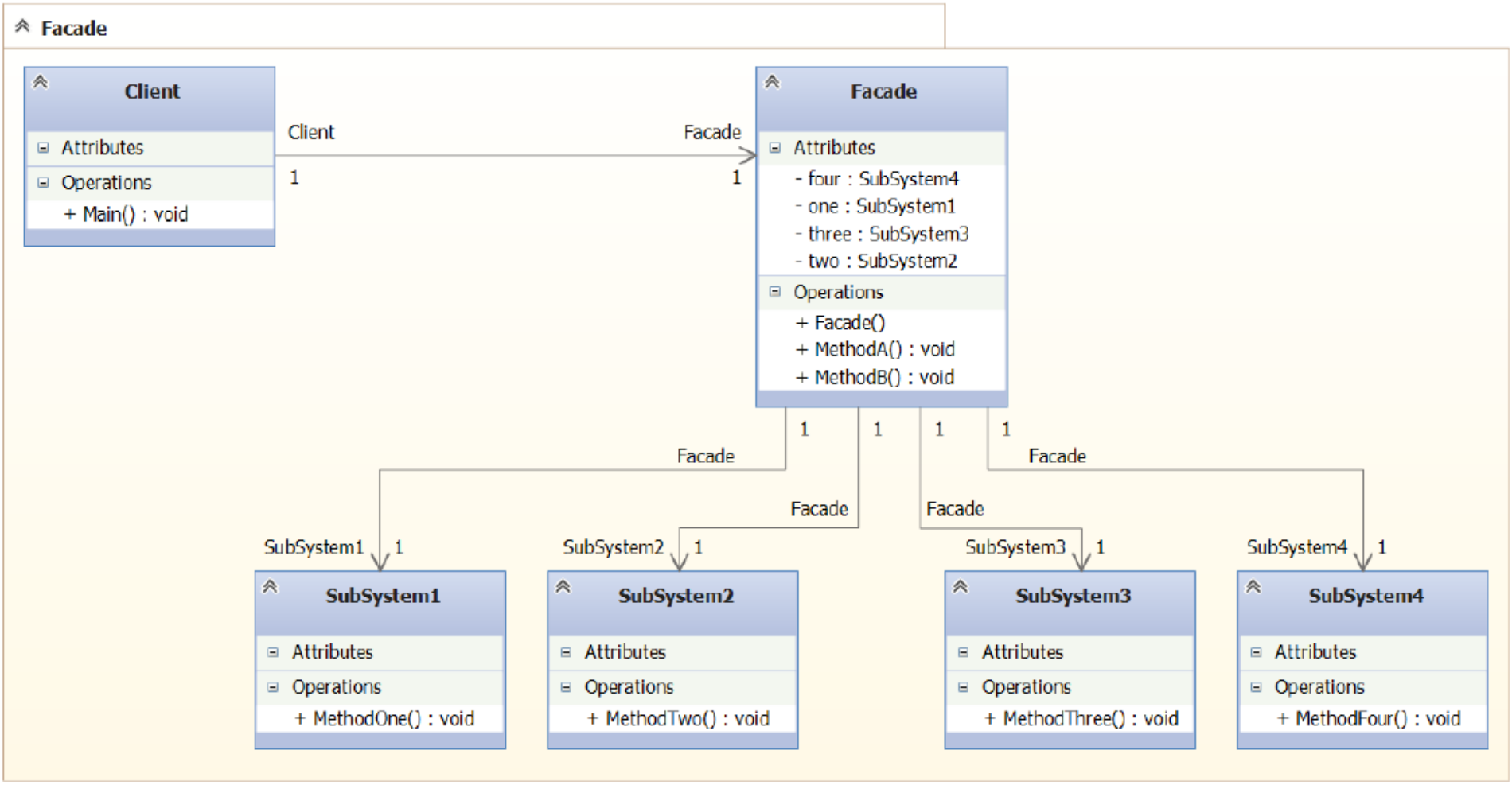
Классы подсистемы не знают о существовании фасада и работают друг с другом напрямую.

Пример

Работа через **Фасад** со сложным фреймворком видеоконвертации



Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Facade facade = new Facade();
        facade.OperationAB();
        facade.OperationBC();
    }
}
```

```
class Facade
{
    SubSystemA subSystemA = new SubSystemA();
    SubSystemB subSystemB = new SubSystemB();
    SubSystemC subSystemC = new SubSystemC();

    public void OperationAB()
    {
        subSystemA.OperationA();
        subSystemB.OperationB();
    }

    public void OperationBC()
    {
        subSystemB.OperationB();
        subSystemC.OperationC();
    }
}
```

Facade - **Фасад**:
Перенаправляет запросы клиентов, другим объектам из которых состоит подсистема.

```
class SubSystemA
{
    public void OperationA()
    {
        Console.WriteLine("SubSystem A");
    }
}
```

```
class SubSystemB
{
    public void OperationB()
    {
        Console.WriteLine("SubSystem B");
    }
}
```

```
class SubSystemC
{
    public void OperationC()
    {
        Console.WriteLine("SubSystem C");
    }
}
```

Classes subsystems – **Классы подсистем**:
Реализуют фактическую функциональность системы. Ничего не знают о фасаде (не имеют ссылок на фасад).

Отношения между участниками

Отношения между классами

- Класс Facade связан связями отношения ассоциации с классами подсистем (Subsystem).

Отношения между объектами

- Клиенты общаются с объектами подсистем, через фасад. Фасад переадресует запросы клиентов подходящим объектам подсистемы.
- Клиенты, которые используют фасад, не должны параллельно работать с объектами подсистем напрямую.

Применение

Когда нужно представить простой или урезанный интерфейс к сложной подсистеме.

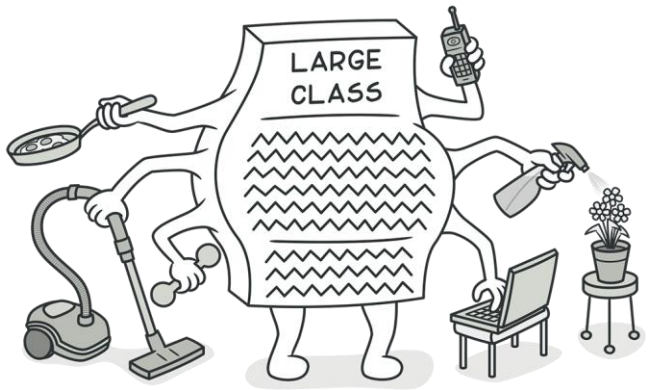
Часто подсистемы усложняются по мере развития программы. Применение большинства паттернов приводит к появлению меньших классов, но в бóльшем количестве. Такую подсистему проще повторно использовать, настраивая её каждый раз под конкретные нужды, но вместе с тем, применять подсистему без настройки становится сложнее. Фасад предлагает определённый вид системы по умолчанию, устраивающий большинство клиентов.

Когда необходимо разделить подсистему на отдельные слои.

Фасады могут быть использованы для определения точек входа на каждый уровень подсистемы. Если подсистемы зависят друг от друга, то зависимость можно упростить, разрешив подсистемам обмениваться информацией только через фасады.

Шаги реализации

1. **Определить, можно ли создать более простой интерфейс, чем тот, который предоставляет сложная подсистема**. Если этот интерфейс избавит клиента от необходимости знать о подробностях подсистемы – решений выбрано верное.
2. **Создать класс фасада, реализующий этот интерфейс**. Он должен переадресовывать вызовы клиента нужным объектам подсистемы. Фасад должен будет позаботиться о том, чтобы правильно инициализировать объекты подсистемы.
3. **Клиент должен работать только с фасадом**. В этом случае изменения в подсистеме будут затрагивать только код фасада, а клиентский код останется рабочим.
4. **Если ответственность фасада** теряет единство, необходимо продумать возможность введения дополнительных фасадов.



Преимущества и недостатки

+ Изолирует клиентов от компонентов сложной подсистемы.

- Фасад рискует стать божественным объектом, привязанным ко всем классам программы.

God object - антипаттерн

